

**DEFINICIÓN DE TECNOLOGÍAS PARA EL DESARROLLO DE ERP. GESTIÓN
DE PROCESOS ACADÉMICOS**
(Módulo de Desarrollo Curricular)

ARQUITECTURA DE SOFTWARE



MARTÍN MONROY RÍOS

AUTORES

MARLON ALMANZA GORDON
FAIBEL DUQUE GIRALDO
CRISTIAN GONZALEZ
ALEJANDRA ROMERO BARROS

UNIVERSIDAD DE CARTAGENA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS

CARTAGENA DE INDIAS D. T. & C
2026

1. Introducción.....	2
2. Metodologías de calificación.....	4
Escala de Calificación.....	4
3. Lenguajes de programación.....	5
3.1. Backend.....	5
3.2. Frontend.....	7
4. Motores de bases de datos.....	10
5. Entornos de desarrollo.....	13
6. Herramientas de versionado.....	15
7. Librerías, API, Frameworks, protocolos.....	17
7.1. Frameworks Backend.....	17
7.2. Frameworks frontend.....	19
7.3. API de integración.....	20
7.4. Librerías.....	22
7.5. Protocolos y seguridad de comunicación.....	24
8. Herramientas para apoyar la implementación de la IU.....	25
9. Herramientas de integración.....	27
Integración CI/CD.....	27
Herramientas Evaluadas.....	27
10. Herramientas para la automatización de pruebas.....	30
Pruebas funcionales y de arquitectura limpia (Unitarias e Integración).....	30
Pruebas no funcionales (Rendimiento, carga y estrés).....	31
11. Herramientas para generación de código con IA.....	33
Bibliografía:.....	35

1. Introducción

En el contexto del proyecto propuesto, orientado al diseño del módulo de Gestión de Procesos Académicos como parte del ERP Académico Academi-CORE, resulta fundamental definir un conjunto de tecnologías que garanticen el correcto desarrollo del currículo académico, su integración con los sistemas existentes y el cumplimiento de las normativas educativas vigentes en Colombia.

El presente documento tiene como propósito establecer y justificar la selección de las tecnologías que soportarán el desarrollo del módulo, tomando como base criterios técnicos, arquitectónicos y de calidad. La elección de dichas tecnologías no se limita únicamente al cumplimiento de los requisitos funcionales, sino que se fundamenta en atributos de calidad definidos en la norma ISO/IEC 25010, tales como rendimiento, seguridad, mantenibilidad, compatibilidad, usabilidad y portabilidad, los cuales son esenciales para asegurar la eficiencia y sostenibilidad del sistema a largo plazo.

También cabe aclarar que el módulo debe integrarse de manera eficiente con otros componentes del ERP, especialmente con el módulo de Gestión Académica y los sistemas institucionales existentes, incluyendo las bases de datos implementadas en Oracle 12C, donde residen datos históricos de estudiantes, asignaturas y estructuras curriculares que deben ser reutilizados y mantenidos.

Adicionalmente, el sistema debe soportar un alto nivel de concurrencia de usuarios, garantizar la disponibilidad de los servicios académicos y asegurar mecanismos de auditoría y control que permitan el seguimiento de cada modificación realizada sobre el currículo, en cumplimiento con las políticas institucionales y los lineamientos del Ministerio de Educación Nacional.

En este sentido, las tecnologías seleccionadas buscan no solo satisfacer los requerimientos operativos del módulo, sino también facilitar su evolución futura dentro de la arquitectura empresarial de la universidad, promoviendo la interoperabilidad, escalabilidad y confiabilidad en la gestión de los procesos académicos.

2. Metodologías de calificación

La metodología de evaluación adoptada en el presente documento se basa en un enfoque comparativo multicriterio, cuyo objetivo es analizar y seleccionar las tecnologías más adecuadas para el desarrollo del sistema propuesto, garantizando coherencia con los requerimientos funcionales y no funcionales del proyecto.

Este proceso se estructura en varias fases. En primer lugar, se identifican las alternativas tecnológicas a evaluar dentro de cada categoría (lenguajes de programación, motores de bases de datos, frameworks, herramientas, entre otros), considerando su relevancia en el contexto actual de desarrollo de software y su aplicabilidad en sistemas empresariales tipo ERP.

Posteriormente, se definen los criterios de evaluación, los cuales corresponden a atributos de calidad del software tales como rendimiento, escalabilidad, seguridad, mantenibilidad, compatibilidad e integración, entre otros. Estos criterios son seleccionados con base en estándares de calidad y buenas prácticas de la industria, asegurando que la evaluación contemple aspectos técnicos y operativos relevantes.

Cada criterio es valorado utilizando una escala ordinal de 1 a 5, donde:

Escala de Calificación

Valor	Interpretación
5	Excelente
4	Alto
3	Medio
2	Bajo
1	Muy bajo

Para cada tecnología evaluada, se asigna un puntaje por criterio sustentado en sus características técnicas, documentación oficial, nivel de adopción en la industria y capacidades comprobadas. Posteriormente, se calcula un puntaje total mediante la suma de

los valores obtenidos en cada criterio, lo que permite establecer una comparación objetiva entre las alternativas analizadas.

Adicionalmente, en algunos casos se consideran ponderaciones implícitas, dando mayor relevancia a criterios críticos para el contexto del proyecto, como la capacidad de manejo de transacciones, seguridad y rendimiento en sistemas empresariales.

Finalmente, con base en los resultados obtenidos, se seleccionan las tecnologías que presentan el mejor desempeño global, las cuales serán utilizadas en las siguientes fases del proceso de diseño e implementación del sistema. Este enfoque garantiza una toma de decisiones fundamentada, transparente y alineada con los objetivos del proyecto.

3. Lenguajes de programación

Se empleó un sistema de evaluación multicriterio ponderado, donde cada lenguaje es clasificado en función de criterios derivados de los atributos de calidad y restricciones del sistema, cada criterio tiene un peso relativo y los lenguajes son evaluados en una escala de 1 a 6. Los criterios fueron definidos a partir de los atributos de calidad del sistema (ISO/IEC 25010), las restricciones tecnológicas (Oracle 12C) y las necesidades de escalabilidad y concurrencia del ERP académico. A continuación se listan los criterios que se utilizaran para la evaluación:

3.1. Backend

Criterio	Descripción	Peso
Rendimiento y manejo de concurrencia	Capacidad para soportar alta carga (hasta 200 TPS) y múltiples usuarios concurrentes sin degradación	20%
Compatibilidad con Oracle 12C	Facilidad de integración con bases de datos existentes y procedimientos almacenados	15%
Escalabilidad	Capacidad de crecer horizontal y verticalmente sin afectar el rendimiento	15%
Seguridad	Soporte para autenticación, autorización, cifrado y auditoría	10%

Mantenibilidad (Clean Architecture)	Facilidad para aplicar arquitectura limpia, bajo acoplamiento y alta cohesión	10%
Ecosistema y frameworks	Disponibilidad de frameworks robustos (Spring, Django, etc.) y librerías empresariales	10%
Facilidad de desarrollo	Productividad, claridad del lenguaje y velocidad de desarrollo	8%
Comunidad y soporte empresarial	Madurez del lenguaje, documentación y respaldo industrial	7%
Facilidad de despliegue	Compatibilidad con contenedores, cloud y CI/CD	5%

Criterio	Peso	Java	C# (.NET)	TypeScript	Python
Rendimiento y concurrencia	20%	5	5	4	2
Compatibilidad con Oracle 12C	15%	5	4	3	3
Escalabilidad	15%	5	5	4	3
Seguridad	10%	5	5	3	3
Mantenibilidad (Clean Architecture)	10%	4	5	4	3
Ecosistema y frameworks	10%	5	5	4	4
Facilidad de desarrollo	8%	2	4	4	5
Comunidad y soporte	7%	5	5	4	5
Facilidad de despliegue	5%	3	4	5	4

Criterio	Peso	Java	C# (.NET)	TypeScript	Python
Rendimiento y concurrencia	0.20	1.00	1.00	0.80	0.40
Compatibilidad con Oracle 12C	0.15	0.75	0.60	0.45	0.45
Escalabilidad	0.15	0.75	0.75	0.60	0.45

Seguridad	0.10	0.50	0.50	0.30	0.30
Mantenibilidad	0.10	0.40	0.50	0.40	0.30
Ecosistema	0.10	0.50	0.50	0.40	0.40
Facilidad de desarrollo	0.08	0.16	0.32	0.32	0.40
Comunidad	0.07	0.35	0.35	0.28	0.35
Despliegue	0.05	0.15	0.20	0.25	0.20

Lenguaje	Puntaje total	Posición
C# (.NET)	4.72	1
Java	4.56	2
TypeScript	3.80	3
Python	3.25	4

El lenguaje **C# (.NET)** obtiene el mayor puntaje (4.72), destacándose por su alta mantenibilidad, seguridad y escalabilidad, además de su fuerte soporte para arquitecturas limpias mediante ASP.NET Core.

Java (4.56) presenta un desempeño muy cercano, sobresaliendo especialmente en la integración con Oracle 12C, lo cual constituye un factor crítico dentro del contexto del sistema ERP académico.

TypeScript (3.80) se posiciona como una alternativa moderna con ventajas en despliegue y desarrollo ágil, aunque con menor robustez en aspectos críticos como seguridad y concurrencia.

Python (3.25), aunque destaca en facilidad de desarrollo, presenta limitaciones en rendimiento y concurrencia, lo que reduce su idoneidad para sistemas de alta demanda transaccional.

3.2. Frontend

Modelo de evaluación multicriterio ponderado, donde cada framework frontend se evalúa con base en atributos derivados de: usabilidad, rendimiento (tiempo de respuesta

< 3s), portabilidad (multi-dispositivo) y mantenibilidad (Clean Architecture / escalabilidad UI)

Criterio	Descripción	Peso
Rendimiento en cliente	Tiempo de carga y respuesta (< 3s), renderizado eficiente	15%
Escalabilidad del frontend	Capacidad de manejar una aplicación grande (ERP modular)	15%
Mantenibilidad y arquitectura	Facilidad para aplicar buenas prácticas (modularidad, Clean Architecture, separación de capas)	15%
Experiencia de usuario (UX)	Facilidad para construir interfaces intuitivas y accesibles	15%
Ecosistema y herramientas	Librerías, UI frameworks, testing, routing, etc.	10%
Facilidad de desarrollo	Curva de aprendizaje y productividad	10%
Comunidad y soporte	Documentación, adopción y respaldo	8%
Integración con backend (APIs)	Facilidad de consumo de servicios REST/GraphQL	7%
Portabilidad (multi-dispositivo)	Adaptabilidad a web, móvil, responsive	5%

Criterio	Peso	Javascript	Typescript	Dart
Rendimiento en navegador	15%	5	5	4
Mantenibilidad y tipado	20%	2	5	5
Escalabilidad del código	15%	3	5	4

Compatibilidad con estándares web	15%	5	5	3
Ecosistema y herramientas	10%	5	5	3
Facilidad de desarrollo	10%	4	4	3
Comunidad y soporte	10%	5	5	3
Integración con backend	5%	5	5	4

Criterio	Peso	JavaScript	TypeScript	Dart
Rendimiento	0.15	0.75	0.75	0.60
Mantenibilidad	0.20	0.40	1.00	1.00
Escalabilidad	0.15	0.45	0.75	0.60
Compatibilidad web	0.15	0.75	0.75	0.45
Ecosistema	0.10	0.50	0.50	0.30
Facilidad desarrollo	0.10	0.40	0.40	0.30
Comunidad	0.10	0.50	0.50	0.30
Integración backend	0.05	0.25	0.25	0.20

Lenguaje	Puntaje total	Posición
TypeScript	4.90	1

JavaScript	4.00	2
Dart	3.75	3

El lenguaje TypeScript obtiene el mayor puntaje (4.90), destacándose principalmente en mantenibilidad, escalabilidad y robustez del código, aspectos fundamentales para el desarrollo de un sistema ERP académico de gran tamaño.

JavaScript (4.00) presenta una alta compatibilidad con los estándares web y un ecosistema ampliamente consolidado, sin embargo, sus limitaciones en tipado y mantenibilidad afectan su idoneidad para aplicaciones complejas.

Por su parte, Dart (3.75) ofrece ventajas en tipado y estructura del código, pero su menor compatibilidad directa con los navegadores y su ecosistema más limitado reducen su aplicabilidad en el desarrollo web empresarial tradicional.

Aunque TypeScript es un superconjunto de JavaScript, su adopción permite mejorar significativamente la calidad del software sin sacrificar compatibilidad, lo que lo convierte en la opción más adecuada para sistemas de gran escala.

4. Motores de bases de datos

Criterio	Descripción	Peso (%)
Compatibilidad con Oracle Database 12c	Capacidad del motor para integrarse, interoperar o reutilizar las bases de datos existentes en Oracle 12c, incluyendo procedimientos almacenados críticos (nómina e historial académico).	14%
Seguridad y Auditoría	Mecanismos de control de acceso, autenticación, autorización y auditoría que permitan cumplir con normativas y garantizar la trazabilidad de las transacciones.	13%

Desempeño	Capacidad del motor para soportar altas tasas de transacciones concurrentes (hasta 200 TPS) y grandes volúmenes de datos (decenas de TB) sin degradación significativa.	12%
Cumplimiento ACID	Garantía de consistencia e integridad de los datos mediante transacciones seguras, especialmente en procesos académicos y administrativos críticos.	11%
Disponibilidad y Recuperación	Capacidad de garantizar disponibilidad del 98% en transacciones, incluyendo mecanismos de replicación, respaldo y recuperación ante fallos.	10%
Escalabilidad	Capacidad de crecer en número de usuarios concurrentes (hasta decenas de miles) y volumen de datos sin afectar el rendimiento.	9%
Integración Tecnológica	Compatibilidad con lenguajes, frameworks, arquitecturas limpias y otros módulos del sistema, permitiendo interoperabilidad dentro del ecosistema ERP.	8%
Costos	Evaluación del costo total de propiedad, incluyendo licencias, operación, mantenimiento e infraestructura.	7%
Facilidad de Administración	Nivel de complejidad en la instalación, configuración, monitoreo y mantenimiento del motor de base de datos.	6%
Adecuación al Modelo de Datos	Capacidad para gestionar estructuras relacionales complejas propias de un ERP (matrículas, notas, usuarios, procesos institucionales).	5%
Comunidad y Soporte	Disponibilidad de documentación, soporte técnico y comunidad activa para resolver problemas y garantizar sostenibilidad.	5%

Criterio	Peso	Oracle database	PostgreSQL	MySQL
Compatibilidad con Oracle 12c	14%	5	3	2
Seguridad y Auditoría	13%	5	4	3

Desempeño	12%	5	4	4
Cumplimiento ACID	11%	5	5	4
Disponibilidad y Recuperación	10%	5	4	4
Escalabilidad	9%	5	4	4
Integración Tecnológica	8%	4	5	5
Costos	7%	2	5	5
Facilidad de Administración	6%	3	4	5
Adecuación al Modelo de Datos	5%	5	5	4
Comunidad y Soporte	5%	4	5	5

Criterio	Peso	Oracle 12c	PostgreSQL	MySQL
Compatibilidad con Oracle 12c	0.14	0.70	0.42	0.28
Seguridad y Auditoría	0.13	0.65	0.52	0.39
Desempeño	0.12	0.60	0.48	0.48
Cumplimiento ACID	0.11	0.55	0.55	0.44
Disponibilidad y Recuperación	0.10	0.50	0.40	0.40
Escalabilidad	0.09	0.45	0.36	0.36
Integración Tecnológica	0.08	0.32	0.40	0.40
Costos	0.07	0.14	0.35	0.35

Facilidad de Administración	0.06	0.18	0.24	0.30
Adecuación al Modelo de Datos	0.05	0.25	0.25	0.20
Comunidad y Soporte	0.05	0.20	0.25	0.25

Motor de Base de Datos	Puntaje Total	Posición
Oracle Database 12c	4.54	1
PostgreSQL	4.22	2
MySQL	3.85	3

A partir de la evaluación ponderada, Oracle Database 12c obtiene el mayor puntaje (4.54%), destacándose principalmente por su compatibilidad con los sistemas existentes, así como por sus capacidades en seguridad, disponibilidad y escalabilidad. En consecuencia, se selecciona como el motor de base de datos más adecuado para el desarrollo del ERP.

5. Entornos de desarrollo

Esto basado en el uso de cada miembro, en soporte en las tecnologías elegidas y el contexto del problema.

Criterio	Descripción	Peso
Productividad del desarrollador	Facilidad para programar, autocompletado, refactorización y herramientas inteligentes.	15%
Soporte para lenguajes	Compatibilidad con Java, C#, JavaScript	12%
Integración con herramientas	Soporte para Git, Docker, bases de datos, testing, etc.	13%
Depuración (Debugging)	Capacidad de detectar y corregir errores eficientemente.	12%

Soporte para arquitectura limpia	Facilidades para organizar código modular y escalable.	10%
Rendimiento del IDE	Consumo de recursos y velocidad del entorno.	10%
Extensibilidad	Plugins y personalización.	10%
Facilidad de uso	Curva de aprendizaje y usabilidad.	10%
Comunidad y soporte	Documentación, foros, actualizaciones.	10%

Criterio	Peso	Visual Studio	IntelliJ IDEA	VS Code	NetBeans
Productividad	0.15	5	5	4	4
Soporte de lenguajes	0.12	5	5	4	4
Integración herramientas	0.13	5	5	4	3
Debugging	0.12	5	4	3	3
Arquitectura limpia	0.10	5	5	3	4
Rendimiento IDE	0.10	4	4	5	3
Extensibilidad	0.08	4	4	5	3
Facilidad de uso	0.10	4	4	5	4
Comunidad y soporte	0.10	5	5	5	4

Criterio	Peso	Visual Studio	IntelliJ IDEA	VS Code	NetBeans
Productividad	0.15	0.75	0.75	0.60	0.60
Soporte lenguajes	0.12	0.60	0.60	0.48	0.48
Integración	0.13	0.65	0.65	0.52	0.39
Debugging	0.12	0.60	0.48	0.36	0.36
Arquitectura limpia	0.10	0.50	0.50	0.30	0.40
Rendimiento	0.10	0.40	0.40	0.50	0.30

Extensibilidad	0.08	0.32	0.32	0.40	0.24
Facilidad uso	0.10	0.40	0.40	0.50	0.40
Comunidad	0.10	0.50	0.50	0.50	0.40

IDE	Puntaje Total (0–5)	Posición
Visual Studio	4.72	1
IntelliJ IDEA	4.60	2
Visual Studio Code	4.16	3
Apache NetBeans	3.57	4

6. Herramientas de versionado

Criterios orientados a la colaboración del equipo, trazabilidad del código, integración con pipelines CI/CD y soporte para el desarrollo modular que exige la arquitectura limpia del proyecto.

Criterio	Descripción	Peso
Colaboración y seguimiento de cambios	Capacidad para gestionar trabajo en equipo, revisión de código mediante pull request, asignación de tareas y trazabilidad de cambios por autor y fecha.	25%
Integración con CI/CD	Compatibilidad nativa o mediante plugins con pipelines de automatización para build, pruebas y despliegue continuo.	22%
Ramificación y fusión	Soporte para gestión avanzada de ramas de desarrollo, estrategias como GitFlow y resolución eficiente de conflictos en la fusión de código.	20%
Historial de cambios	Capacidad para registrar, consultar y revertir el historial complejo de modificaciones del código fuente de forma confiable.	15%

Conocimiento del equipo	Nivel de familiaridad y experiencia previa del equipo con la herramienta, reduciendo tiempos de adopción y errores operativos.	10%
Facilidad de uso	Intuitividad de la interfaz, disponibilidad de documentación clara y accesibilidad para integrarse con distintos niveles de experiencia.	8%

Criterio	Peso	Git (GitHub)	GitLab	Bitbucket
Colaboración y seguimiento de cambios	25%	5	5	4
Integración con CI/CD	22%	5	5	3
Ramificación y fusión	20%	5	5	4
Historial de cambios	15%	5	5	4
Conocimiento del equipo	10%	5	3	3
Facilidad de uso	8%	4	4	4

Criterio	Peso	JavaScript	TypeScript	Dart
Rendimiento	0.25	1.25	1.25	1.0
Mantenibilidad	0.22	1.1	1.1	0.66
Escalabilidad	0.20	1.0	1.0	0.75
Compatibilidad web	0.15	0.75	0.75	0.60

Ecosistema	0.10	0.50	0.30	0.30
Facilidad desarrollo	0.08	0.32	0.32	0.32

Lenguaje	Puntaje total
Git	4.92
GitLab	4.72
BitBucket	3.74

La elección ha sido **Git con GitHub** — es la plataforma de versionado más usada a nivel mundial, con integración nativa con GitHub Actions para CI/CD, soporte avanzado de ramas, pull requests y revisión de código colaborativa. El equipo ya tiene familiaridad con ella, lo que reduce tiempos de adopción y errores operativos.

7. Librerías, API, Frameworks, protocolos

7.1. Frameworks Backend

Criterio	Peso	Descripción
Rendimiento y eficiencia	20%	Capacidad de responder rápido, manejar carga concurrente y consumir pocos recursos
Escalabilidad	15%	Facilidad para crecer en usuarios, servicios y volumen de transacciones sin rediseñar la base técnica
Seguridad	15%	Soporte para autenticación, autorización, hardening, validación y control de acceso
Ecosistema y madurez	15%	Nivel de comunidad, documentación, librerías, estabilidad y soporte empresarial
Integración con APIs y bases de datos	15%	Facilidad para consumir/exponer REST y conectarse con motores SQL y otros sistemas institucionales
Mantenibilidad y soporte a largo plazo	10%	Claridad estructural, pruebas, modularidad, actualización y sostenibilidad del código

Desarrollo móvil	10%	Rapidez con la que el equipo puede desarrollar, probar y evolucionar el backend
------------------	-----	---

Criterio	Peso	Spring Boot	ASP.NET Core (C#)	NestJS
Rendimiento y eficiencia	20%	5	5	4
Escalabilidad	15%	5	5	4
Seguridad	15%	5	5	4
Ecosistema y madurez	15%	5	4	4
Integración con APIs y bases de datos	15%	5	5	4
Mantenibilidad y soporte a largo plazo	10%	5	4	4
Curva de aprendizaje y productividad	10%	4	4	5

Criterio	Peso	Spring Boot	ASP.NET Core (C#)	NestJS
Rendimiento y eficiencia	0.20	1.00	1.00	0.80
Escalabilidad	0.15	0.75	0.75	0.60
Seguridad	0.15	0.75	0.75	0.60
Ecosistema y madurez	0.15	0.75	0.60	0.60
Integración con APIs y bases de datos	0.15	0.75	0.75	0.60
Mantenibilidad y soporte a largo plazo	0.10	0.50	0.40	0.40
Curva de aprendizaje y productividad	0.10	0.40	0.40	0.50

Framework	Puntuación Total	Posición
-----------	------------------	----------

Spring Boot	4.9	1°
ASP.NET core (C#)	4.65	2°
NestJS	4.1	3°

El framework Spring Boot presenta un mejor ajuste global para un backend institucional orientado a transacciones, seguridad, escalabilidad e integración.

Por su lado, ASP.NET es una alternativa de muy alto nivel, especialmente si se prioriza rendimiento y el equipo ya que domina C#.

Y finalmente NestJS es una buena opción para velocidad de desarrollo, pero menos fuerte para el nivel crítico y madurez requeridas por el ERP.

7.2. Frameworks frontend

Criterio	Peso	Descripción
Ecosistema y compatibilidad	20%	Capacidad de integrarse con librerías UI, routing, testing y herramientas del entorno web.
Madurez y comunidad	20%	Nivel de adopción, documentación, estabilidad y soporte de la comunidad.
Rendimiento y escalabilidad	20%	Capacidad para mantener buen desempeño en una aplicación modular y grande.
Mantenimiento a largo plazo	15%	Facilidad para sostener, extender y reorganizar el código con el tiempo.
Curva de aprendizaje	10%	Facilidad para que el equipo adopte la tecnología con rapidez.
Conocimiento del equipo	10%	Ajuste con la experiencia previa del grupo de trabajo.
Desarrollo móvil	5%	Capacidad de extenderse a escenarios responsive o multiplataforma.

Criterio	Peso	React	Angular	Vue.js
Ecosistema y compatibilidad	20%	5	4	3
Madurez y comunidad	20%	5	5	4

Rendimiento y escalabilidad	20%	4	3	4
Mantenimiento a largo plazo	15%	4	5	3
Curva de aprendizaje	10%	3	2	4
Conocimiento del equipo	10%	4	3	2
Desarrollo Móvil	5%	4	3	3

Criterio	Peso	React	Angular	Vue.js
Ecosistema y compatibilidad	20%	1.00	0.80	0.60
Madurez y comunidad	20%	1.00	1.00	0.80
Rendimiento y escalabilidad	20%	0.80	0.60	0.80
Mantenimiento a largo plazo	15%	0.60	0.75	0.45
Curva de aprendizaje	10%	0.30	0.20	0.40
Conocimiento del equipo	10%	0.40	0.30	0.20
Desarrollo Móvil	5%	0.20	0.15	0.15

Framework	Puntuación Total	Posición
React	4.3	1°
Angular	3.8	2°
Vue.js	3.45	3°

React queda arriba porque combina un ecosistema muy maduro, buena escalabilidad y un balance sólido entre rendimiento y mantenimiento.

Angular conserva una posición fuerte por su estructura empresarial, pero su curva de aprendizaje y su menor flexibilidad práctica lo dejan detrás.

Vue.js aparece tercero por ser liviano y fácil de adoptar, aunque con menor tracción ecosistémica para un ERP grande.

7.3. API de integración

Criterio	Peso	Descripción
Interoperabilidad con otros módulos	25%	Facilidad para integrarse con otros componentes internos del ERP.
Adecuación al contexto del ERP académico	20%	Ajuste al tipo de comunicación que necesita el sistema académico.
Simplicidad de consumo e implementación	20%	Facilidad de uso para consumir o exponer servicios.
Seguridad e integración con autenticación	15%	Compatibilidad con mecanismos de seguridad y control de acceso.
Soporte de herramientas y documentación	10%	Nivel de madurez, librerías y soporte disponible.
Mantenibilidad a largo plazo	10%	Facilidad para sostener la solución en el tiempo.

Criterio	Peso	REST/ JSON	GraphQL	SOAP
Interoperabilidad con otros módulos	25%	5	3	2
Adecuación al contexto del ERP académico	20%	5	3	2
Simplicidad de consumo e implementación	20%	5	3	1
Seguridad e integración con autenticación	15%	5	4	3
Soporte de herramientas y documentación	10%	5	3	2
Mantenibilidad a largo plazo	10%	5	4	2

Criterio	Peso	React	Angular	Vue.js
Interoperabilidad con otros módulos	25%	1.25	0.75	0.50
Adecuación al contexto del ERP académico	20%	1.00	0.60	0.40
Simplicidad de consumo e implementación	20%	1.00	0.60	0.20

Seguridad e integración con autenticación	15%	0.75	0.60	0.45
Soporte de herramientas y documentación	10%	0.50	0.30	0.20
Mantenibilidad a largo plazo	10%	0.50	0.40	0.20

Framework	Puntuación Total	Posición
REST/JSON	5.0	1º
GraphQL	3.25	2º
SOAP	1.95	3º

REST/JSON es la mejor opción porque resuelve con menos fricción la interoperabilidad entre módulos del ERP, además de ser simple de consumir, muy compatible con el enfoque web y suficientemente maduro.

GraphQL aporta flexibilidad, pero añade complejidad operativa.

SOAP queda último por ser más pesado y menos natural para una arquitectura moderna basada en web services.

7.4. Librerías

Criterio	Peso	Descripción
Compatibilidad con React y TypeScript	20%	Qué tan bien se integra la librería con el stack frontend definido.
Utilidad funcional para el módulo	20%	Valor práctico real para resolver necesidades del ERP.
Mantenimiento y comunidad	15%	Madurez, soporte y continuidad del proyecto.
Curva de aprendizaje	15%	Facilidad para que el equipo la adopte rápidamente.

Rendimiento/ Ligereza	15%	Consumo de recursos y sobrecarga sobre la aplicación.
Cobertura de casos académicos	15%	Capacidad de resolver escenarios frecuentes del módulo.

Criterio	Peso	Axios	React Hook Form	Zod	date-fns
Compatibilidad con React y TypeScript	20%	5	5	5	5
Utilidad funcional para el módulo	20%	5	5	4	5
Mantenimiento y comunidad	15%	5	5	4	5
Curva de aprendizaje	15%	4	5	4	5
Rendimiento/ Ligereza	15%	4	5	4	4
Cobertura de casos académicos	15%	4	4	4	4

Criterio	Peso	Axios	React Hook Form	Zod	date-fns
Compatibilidad con React y TypeScript	0.20	1.00	1.00	1.00	1.00
Utilidad funcional para el módulo	0.20	1.00	1.00	0.80	1.00
Mantenimiento y comunidad	0.15	0.75	0.75	0.60	0.75
Curva de aprendizaje	0.15	0.60	0.75	0.60	0.75
Rendimiento/ Ligereza	0.15	0.60	0.75	0.60	0.60
Cobertura de casos académicos	0.15	0.60	0.60	0.60	0.60

Framework	Puntuación Total	Posición
React Hook Form	4.85	1°
date-fns	4.70	2°
Axios	4.55	3°
Zod	4.20	4°

React Hook Form y date-fns destacan porque resuelven problemas concretos del módulo con bajo costo cognitivo y buena integración con React/TypeScript sin embargo resalto mas React Hook Form por su alta curva de aprendizaje.

Axios se mantiene alto por su confiabilidad en consumo HTTP.

Zod conserva una valoración muy buena, aunque un poco más limitada por su foco específico en validación de esquemas.

7.5. Protocolos y seguridad de comunicación

Criterio	Peso	Descripción
Seguridad de transporte	30	Protección del canal de comunicación frente a interceptación o manipulación.
Compatibilida Web	20	Facilidad de uso en entornos web modernos y navegadores.
Facilidad de implementación	20	Esfuerzo técnico necesario para adoptarlo correctamente.
Integración con API REST	15	Qué tan natural es su uso con servicios REST del ERP.
Madurez y estandarización	15	Nivel de adopción como estándar confiable.

Criterio	Peso	HTTPS	OAuth	WebSocket
Seguridad de transporte	30%	5	5	2
Compatibilida Web	20%	5	4	3
Facilidad de implementación	20%	5	4	3

Integración con API REST	15%	5	5	3
Madurez y estandarización	15%	5	5	3

Criterio	Peso	HTTPS	OAuth	WebSocket
Seguridad de transporte	30%	1.50	1.50	0.60
Compatibilida Web	20%	1.00	0.80	0.45
Facilidad de implementación	20%	1.00	0.80	0.45
Integración con API REST	15%	0.75	0.75	0.45
Madurez y estandarización	15%	0.75	0.75	0.45

Framework	Puntuación Total	Posición
HTTPS	5.00	1º
OAuth 2.0	4.60	2º
WebSocket	2.40	3º

HTTPS lidera porque aporta seguridad de transporte de forma estándar y universal.

OAuth 2.0 queda muy cerca por su valor en autenticación y autorización.

WebSocket es útil en tiempo real, pero para este módulo no parece tan crítico como los otros dos protocolos, por eso recibe una valoración menor.

8. Herramientas para apoyar la implementación de la IU

Criterios orientados a la automatización del pipeline de desarrollo, tolerancia a fallos (requisito del proyecto: recuperación en menos de 5 segundos para fallos autogestionables), trazabilidad y compatibilidad con GitHub ya seleccionado.

Criterio	Descripción	Peso
Compatibilidad con React	Soporte nativo o mediante integración directa con React, sin configuraciones adicionales complejas.	28%

Personalización de estilos institucionales	Capacidad para aplicar y mantener colores, tipografías y elementos visuales propios de la institución.	22%
Facilidad para implementar interfaces intuitivas	Disponibilidad de componentes accesibles, listos para usar, que reduzcan el tiempo de aprendizaje del usuario final.	20%
Integración diseño-implementación	Compatibilidad con herramientas de diseño como Figma para facilitar la transición entre prototipo y código.	15%
Documentación y comunidad	Calidad y disponibilidad de recursos, tutoriales y soporte de la comunidad.	10%
Costo / Licencia	Disponibilidad de versión gratuita o de código abierto sin restricciones para uso académico o empresarial.	5%

Criterio	Peso	Tailwind CSS + shadcn/ui	UI	Design
Colaboración y seguimiento de cambios	28%	5	5	5
Integración con CI/CD	22%	5	4	3
Ramificación y fusión	20%	5	4	4
Historial de cambios	15%	5	4	3
Conocimiento del equipo	10%	4	5	5
Facilidad de uso	5%	5	5	5

Criterio	Peso	Tailwind CSS + shadcn/ui	Material UI	Ant Design
----------	------	--------------------------	-------------	------------

Rendimiento	0.28	1.4	1.4	1.4
Mantenibilidad	0.22	1.1	0.88	0.66
Escalabilidad	0.20	1.0	0.8	0.8
Compatibilidad web	0.15	0.75	0.6	0.45
Ecosistema	0.10	0.4	0.5	0.5
Facilidad desarrollo	0.05	0.25	0.25	0.25

Lenguaje	Puntaje total
Tailwind	4.95
MaterialUI	4.43
Ant Design	4.00

Tailwind CSS + shadcn/ui — su compatibilidad nativa con React, control total sobre la identidad institucional mediante clases utilitarias, componentes accesibles listos para usar y excelente integración con Figma lo convierten en la opción más sólida. Al ser código abierto y gratuito, también elimina cualquier restricción de licenciamiento para el proyecto.

9. Herramientas de integración

Integración CI/CD

Criterio	Justificación	Peso
Automatización	Criterio más importante para CI/CD efectivo	25%
Interoperabilidad	Esencial para integración con ecosistema existente	20%
Facilidad de integración	Reduce tiempo de implementación y curva de aprendizaje	18%
Tolerancia a fallos	Garantiza continuidad del pipeline	15%
Logs y monitoreo	Importante para debugging y observabilidad	12%

Seguridad	Configurable según necesidades específicas	8%
Precios y documentación	Factor secundario con herramientas open source	2%

Herramientas Evaluadas

Jenkins

- Logs y monitoreo: Sistema de logs completo con plugins como Blue Ocean, integración con ELK Stack, métricas detalladas de builds y pipelines
- Tolerancia a fallos: Excelente tolerancia con reintentos automáticos, recuperación de builds fallidos, backup y restore automático
- Interoperabilidad: Máxima interoperabilidad con +1800 plugins, APIs REST, integración con cualquier herramienta del mercado
- Seguridad: Sistema robusto de seguridad con roles granulares, autenticación LDAP/OAuth, configurable según necesidades
- Automatización: Automatización completa con pipelines declarativos e imperativos, triggers avanzados, scheduled builds
- Precios y documentación: Gratuito y open source, documentación extensa y comunidad activa
- Facilidad de integración: Flexible y bien documentado, gran cantidad de ejemplos y casos de uso

Docker

- Logs y monitoreo: Logs nativos de contenedores, fácil integración con herramientas de monitoreo estándar
- Tolerancia a fallos: Excelente tolerancia con restart policies, health checks, contenedores inmutables
- Interoperabilidad: Máxima interoperabilidad, funciona con cualquier aplicación y plataforma, estándar de la industria
- Seguridad: Aislamiento por contenedores, image scanning, security policies configurables
- Automatización: Automatización completa de empaquetado, distribución y despliegue, integración perfecta con CI/CD
- Precios y documentación: Gratuito para uso empresarial, documentación excelente y clara
- Facilidad de integración: Extremadamente fácil de integrar, Dockerfile simple, adoptado universalmente

GitLab

- Logs y monitoreo: Sistema integrado de logs y monitoreo, métricas built-in, integración con Prometheus nativo

- Tolerancia a fallos: Buena tolerancia con retry policies, pipeline recovery, distributed runners
- Interoperabilidad: Buena interoperabilidad con APIs REST, webhooks, pero más limitada fuera del ecosistema GitLab
- Seguridad: Seguridad empresarial robusta, SAST/DAST integrado, dependency scanning, compliance frameworks
- Automatización: Automatización completa con GitLab CI/CD, Auto DevOps, scheduled pipelines, GitOps workflows
- Precios y documentación: Plan gratuito limitado, planes paid escalables, documentación excelente
- Facilidad de integración: Fácil si se usa ecosistema GitLab completo, más compleja con herramientas externas

ArgoCD

- Logs y monitoreo: Logs detallados de sync operations, UI rica para monitoreo de aplicaciones, métricas de Prometheus
- Tolerancia a fallos: Excelente tolerancia con self-healing, rollback automático, health checks continuos
- Interoperabilidad: Interoperabilidad limitada a Kubernetes, excelente dentro de este ecosistema específico
- Seguridad: Seguridad robusta con RBAC, SSO integration, Git repository access controls, encryption at rest
- Automatización: Automatización GitOps completa, continuous sync, declarative deployments, automated rollbacks
- Precios y documentación: Completamente gratuito y open source, documentación técnica especializada
- Facilidad de integración: Requiere Kubernetes y conocimiento específico de GitOps patterns

Criterio	Peso	Jenkins	Docker	GitLab	ArgoCD
Automatización	25	0.95	0.90	0.85	0.85
Interoperabilidad	20	0.95	0.95	0.75	0.60
Facilidad de integración	18	0.85	0.95	0.80	0.65
Tolerancia a fallos	15	0.90	0.85	0.80	0.90
Logs y monitoreo	12	0.90	0.75	0.85	0.85
Seguridad	8	0.80	0.75	0.90	0.85
Precios y documentación	2	0.90	0.95	0.75	0.85

Herramienta	Puntuación Total	Posición
Jenkins	90.25 - 4.51	1°
Docker	89.50 - 4.48	2°
GitLab	81.25 - 4.06	3°
ArgoCD	76.75 - 3.84	4°

Recomendación Final: Jenkins + Docker

10. Herramientas para la automatización de pruebas

Para garantizar una cobertura integral de los atributos de calidad exigidos en el sistema ERP académico, el análisis de herramientas se dividió en dos frentes arquitectónicos: la validación funcional y la validación no funcional.

Pruebas funcionales y de arquitectura limpia (Unitarias e Integración)

Criterio	Descripción	Peso
Aislamiento y Mocking (Oracle 12C)	Capacidad nativa para simular dependencias externas y bases de datos heredadas sin alterar datos.	25%
Mantenibilidad (Arquitectura Limpia)	Facilidad para probar capas de forma aislada respetando el bajo acoplamiento.	20%
Ecosistema y Herramientas	Disponibilidad de reportes de cobertura de código y aserciones integradas.	15%
Facilidad de Desarrollo	Curva de aprendizaje, escritura de código limpio y legibilidad de los scripts.	15%
Rendimiento en Ejecución	Velocidad de ejecución de las suites de pruebas en entornos locales.	15%
Comunidad y Soporte	Documentación oficial, adopción en la industria y resolución de problemas.	10%

Criterio	Peso	Jest	Vitest	Mocha
Aislamiento y Mocking (Oracle 12C)	25%	5	5	3
Mantenibilidad (Arquitectura Limpia)	20%	5	5	4
Ecosistema y Herramientas	15%	5	4	4
Facilidad de Desarrollo	15%	4	4	3
Rendimiento en Ejecución	15%	4	5	4
Comunidad y Soporte	10%	5	4	5

Criterio	Peso	Jest	Vitest	Mocha
Aislamiento y Mocking (Oracle 12C)	0.25	1.25	1.25	0.75
Mantenibilidad (Arquitectura Limpia)	0.20	1.00	1.00	0.80
Ecosistema y Herramientas	0.15	0.75	0.60	0.60
Facilidad de Desarrollo	0.15	0.60	0.60	0.45
Rendimiento en Ejecución	0.15	0.60	0.75	0.60
Comunidad y Soporte	0.10	0.50	0.40	0.50

Lenguaje	Puntaje total
Jest	4.70
Vitest	4.60
Mocha	3.70

Justificación: La herramienta Jest obtiene el mayor puntaje (4.70). Sobresale por su ecosistema maduro y su capacidad sobresaliente para emular (mocking) las bases de datos de trabajadores y estudiantes en Oracle 12C , garantizando que la arquitectura limpia se mantenga verificable sin dependencias externas rígidas.

Pruebas no funcionales (Rendimiento, carga y estrés)

Criterio	Descripción	Peso
Generación de Carga (200 TPS)	Capacidad del motor para sostener hasta 200 transacciones concurrentes por segundo.	30%
Consumo de Recursos (Eficiencia)	Uso de CPU y memoria RAM al simular miles de usuarios concurrentes.	20%
Mantenibilidad de Scripts	Facilidad para programar los escenarios de prueba utilizando lenguajes modernos y limpios.	20%
Métricas y Reportes	Trazabilidad para validar tiempos de respuesta inferiores a los 3 segundos requeridos.	15%
Comunidad y Ecosistema	Integración con herramientas de CI/CD y visualización de datos (ej. Grafana).	15%

Criterio	Peso	Grafana K6	Apache JMeter	Gatling
Generación de Carga (200 TPS)	30%	5	5	5
Consumo de Recursos (Eficiencia)	20%	5	2	4
Mantenibilidad de Scripts	20%	5	2	3
Métricas y Reportes	15%	4	5	4
Comunidad y Ecosistema	15%	4	5	3

Criterio	Peso	Grafana K6	Apache JMeter	Gatling
Generación de Carga (200 TPS)	0.30	1.50	1.50	1.50

Consumo de Recursos (Eficiencia)	0.20	1.00	0.40	0.80
Mantenibilidad de Scripts	0.20	1.00	0.40	0.60
Métricas y Reportes	0.15	0.60	0.75	0.60
Comunidad y Ecosistema	0.15	0.60	0.75	0.45

Lenguaje	Puntaje total
Grafana K6	4.70
Apache JMeter	3.8
Gatling	3.95

Justificación: La herramienta **Grafana K6** se posiciona en el primer lugar (4.70). Aunque JMeter posee excelentes capacidades de reporte, K6 maximiza la eficiencia en el consumo de recursos de hardware al momento de simular las 200 transacciones concurrentes por segundo exigidas para la gestión de procesos académicos. Además, sus scripts se redactan en código, facilitando el cumplimiento de las normativas de programación limpia.

11. Herramientas para generación de código con IA

La integración con de asistentes de Inteligencia Artificial en el ciclo de desarrollo requiere un análisis estricto frente a los criterios de evaluación del proyecto, específicamente en lo concerniente al "Respeto a los derechos de autor" y la "Originalidad del material". El uso de herramientas no controladas presenta el riesgo de filtrar la lógica de negocio del ERP institucional a modelos de entrenamiento públicos, o de introducir código de terceros con licencias restrictivas sin la debida atribución.

Criterio	Descripción	Peso
Privacidad y Derechos de Autor	Capacidad de bloquear la telemetría para evitar que el código de la institución alimente modelos públicos de IA.	35%

Comprensión de Arquitectura Limpia	Capacidad de analizar el contexto completo del proyecto en el IDE para sugerir código que respete la separación de capas.	25%
Efectividad en Código Limpio	Calidad de las sugerencias generadas y su alineación con buenas prácticas de programación y refactorización.	20%
Filtrado de Licencias de Terceros	Capacidad para detectar e impedir la sugerencia de fragmentos de código protegidos por licencias restrictivas de código abierto.	10%
Integración con el Flujo de Trabajo	Compatibilidad nativa con el entorno de desarrollo (Visual Studio Code) y herramientas de versionado.	10%

Criterio	Peso	Github Copilot	Amazon Developer	Tabnine	ChatGPT (web)
Privacidad y Derechos de Autor	35%	5	5	5	1
Comprensión de Arquitectura Limpia	25%	5	4	3	2
Efectividad en Código Limpio	20%	5	4	4	4
Filtrado de Licencias	10%	4	5	3	1
Integración con el Flujo de Trabajo	10%	5	5	4	1

	Peso	Github Copilot	Amazon Developer	Tabnine	ChatGPT (web)
Privacidad y Derechos de Autor	35%	1.75	1.75	1.75	0.35
Comprensión de Arquitectura Limpia	25%	1.25	1.00	0.75	0.50
Efectividad en Código Limpio	20%	1.00	0.80	0.80	0.80

Filtrado de Licencias	10%	0.40	0.50	0.30	0.10
Integración con el Flujo de Trabajo	10%	0.50	0.50	0.40	0.10

Lenguaje	Puntaje total
Github Copilot	4.90
Amazon Q Developer	4.55
Tabnine	4.00
ChatGPT	1.85

Justificación: Con un puntaje destacado de 4.90, se selecciona **GitHub Copilot**. La herramienta cumple a cabalidad con la restricción de "evidenciar la programación de código limpio" al estar profundamente integrada con el contexto semántico del IDE. A diferencia de soluciones basadas en web como ChatGPT (cuya puntuación es penalizada severamente por comprometer la confidencialidad de la lógica de negocio al externalizar los datos), Copilot permite a la organización desactivar la telemetría, garantizando que el ERP académico no se utilice para entrenar modelos públicos. Amazon Q Developer representa una excelente alternativa corporativa gracias a su estricto control de licencias, pero Copilot ofrece una comprensión más madura de las estructuras de "arquitectura limpia" requeridas para el módulo de Desarrollo del Currículo.

Bibliografía:

Emadamerho-Atori, N. (2024, octubre 7). Angular vs. React vs. Vue.js: Comparing performance. LogRocket Blog. <https://blog.logrocket.com/angular-vs-react-vs-vue-js-comparing-performance/>

Putra, A. [@AntonPutra]. (s/f). MySQL vs PostgreSQL performance benchmark (latency - throughput - saturation). Youtube. Recuperado el 10 de junio de 2025, de  MySQL vs PostgreSQL Performance Benchmark (Latency - Throughput - Saturation)

Putra, A. [@AntonPutra]. (s/f-b). Quarkus (java) vs fiber (go): Performance benchmark in kubernetes #201. Youtube. Recuperado el 10 de junio de 2025, de <https://www.youtube.com/watch?v=PL0c-SvjSVg>

Writtify. (2024, junio 21). JavaScript vs TypeScript: Choosing the right language for developers in 2024. Writtify. <https://writtify.com/javascript-vs-typescript-choosing-the-right-language-for-developers-in-2024/>

Acerca de GitHub y Git. (s/f). Recuperado de: <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>

Selecting the right software development tools for your developers. (s/f). Smartdraw.com. Recuperado el 10 de junio de 2025, de <https://www.smartdraw.com/technology/right-software-development-tools-for-developers.htm>

About Katalon Studio. (s/f). Katalon.com. Recuperado el 10 de junio de 2025, de <https://docs.katalon.com/katalon-studio/about-katalon-studio>

